



DEPL Push Gateway

운영·연동·장애 대응 가이드

구성	Spring Boot 3.5 · Java 21 · MariaDB · Firebase Cloud Messaging
전달	DB Outbox · FCM Topic · APNs/iOS · Android
보안	폐쇄망 outbound HTTPS 443 · 서명 QR · 업무 상세 미전송

DEPL · Push notification portfolio

Spring Boot 3.5 / Java 21 / MariaDB outbox를 사용해 FCM Topic으로 고정 8종 알림을 보내는 폐쇄망 Gateway입니다. 앱은 Gateway에 접속하지 않습니다. 업무 화면에서 서명된 QR을 촬영해 개인·부서·공지 Topic을 구독하고, Gateway만 Google FCM에 outbound HTTPS 443으로 접속합니다.

통신과 처리 구조

```
업무 트랜잭션 → push_queue → 30초 Scheduler → Topic binding 해석
→ Firebase Admin setTopic → FCM → iPhone
→ FCM_ACCEPTED history INSERT + queue DELETE
```

```
업무 화면 → ECDSA 서명 QR → iPhone → 개인/부서/notice_all 구독
```

FCM_ACCEPTED는 Firebase가 요청을 접수했다는 의미이며 단말 표시 완료를 보장하지 않습니다. 재시도는 30초 → 2분 → 5분 → 15분이며 영구 오류, 인증 오류, Topic 누락/비활성은 무한 재시도하지 않고 DEAD가 됩니다. FOR UPDATE SKIP LOCKED, stale PROCESSING 복구, history INSERT와 queue DELETE의 원자성은 기존과 같습니다.

V2 Topic 마이그레이션

V2_topic_delivery.sql은 V1을 수정하지 않고 다음 작업을 수행합니다.

- push_topic_binding과 NOTICE / ALL / notice_all 생성
- queue/history에 target_type, target_id, delivery_channel=TOPIC 추가
- 기존 recipient_user_id를 USER 대상에 backfill
- 대상-상태-시간 기반 인덱스 및 unique/check 제약 추가
- 호환 기간을 위해 recipient_user_id, push_device, 기존 단말 API 유지

배포 순서는 DB 백업 → V2 적용 → 새 생산 코드 배포 → Gateway 배포 → QR 재등록입니다. 롤백은 생산/스케줄러를 먼저 중지하고 미처리 큐를 확인한 뒤 DBA 승인을 받아

db/rollback/V2_rollback_topic_delivery.sql을 수동 적용합니다. 새 코드에서 만들어진 DEPARTMENT/NOTICE 행은 구버전이 이해하지 못하므로 코드만 단독 롤백하면 안 됩니다.

push_device와 /api/push/devices는 deprecated 호환 요소이며 Topic 운영 안정화 후 별도 배포에서 생산/조회 여부 확인 → 백업 → API 제거 → 테이블 제거 순으로 정리합니다.

환경 변수

변수	기본값	설명
DB_URL	jdbc:mariadb://localhost:3306/business	업무 DB와 같은 MariaDB DataSource
DB_USERNAME / DB_PASSWORD	business / 빈 값	DB 계정
PUSH_SCHEDULER_ENABLED	true	30초 Queue worker
PUSH_SCHEDULER_DELAY	30s	조회 간격
PUSH_BATCH_SIZE	100	1회 선점 건수
PUSH_PROCESSING_TIMEOUT	5m	stale PROCESSING 기준
PUSH_MAX_ATTEMPTS	5	최대 시도 횟수
FIREBASE_ENABLED	false	실제 FCM 전송
FIREBASE_PROJECT_ID	빈 값	iOS 앱과 같은 Project ID
GOOGLE_APPLICATION_CREDENTIALS	없음	Secret mount의 서비스 계정 JSON 경로
PUSH_QR_PRIVATE_KEY_PATH	없음	PKCS#8 P-256 개인키 PEM 경로

변수	기본값	설명
PUSH_QR_TTL	3m	QR 유효시간, 최대 10분
PUSH_TEST_PAGE_ENABLED	false	내부 테스트 화면 활성화
PUSH_TEST_PAGE_AUTHENTICATION_ENABLED	true	테스트 화면 로그인 사용 여부
PUSH_TEST_PAGE_USERNAME	없음	테스트 관리자 계정
PUSH_TEST_PAGE_PASSWORD_HASH	없음	BCrypt hash만 허용
PUSH_TEST_ENQUEUE_WHEN_FIREBASE_DISABLED	true	FCM off 시 Queue 등록 여부
PUSH_TEST_REQUESTS_PER_MINUTE	10	관리자별 분당 발송 제한

서비스 계정 JSON, 개인키, 비밀번호 원문은 저장소·DB·이미지·로그에 넣지 않습니다.

로컬 실행

필요 도구는 JDK 21, Maven, MariaDB 10.6 이상입니다. 환경변수 이름은 `.env.example`을 참고하되 실제 값을 저장소 안의 `.env`에 보관하지 마세요.

```
mvn test
mvn package
java -jar target/push-gateway-0.1.0-SNAPSHOT.jar
```

실행 환경에는 최소한 DB 접속정보가 필요합니다. 실제 FCM과 QR을 사용하려면 `FIREBASE_ENABLED`, Firebase Project ID, 서비스 계정 경로, QR 개인키 경로를 함께 설정합니다. 관리자 화면을 활성화할 때는 다음처럼 BCrypt hash를 생성해 Secret으로 주입합니다.

```
htpasswd -bnBC 12 "" "choose-a-strong-password"
```

출력에서 앞의 구분 문자만 제거한 전체 `$2y$...` 문자열을 `PUSH_TEST_PAGE_PASSWORD_HASH`로 사용합니다. 셸에서 `$`가 변수로 해석되지 않도록 Secret Manager, IDE 환경 설정 또는 안전한 env-file 로더를 사용하세요.

Firebase와 QR 키 설정

Firebase Console에서 DEPL iOS 앱과 APNs 인증키를 연결하고 전용 최소권한 서비스 계정 JSON을 Secret으로 마운트합니다. Gateway에서 필요한 외부 통신은 Google 인증/FCM/DNS 방향 outbound TCP 443뿐이며 신규 inbound 연결은 필요 없습니다.

Firebase 서버 발송 설정

1. Firebase 프로젝트에서 Cloud Messaging API(HTTP v1)를 활성화합니다.
2. Gateway 전용 서비스 계정을 만들고 필요한 최소 권한만 부여합니다.
3. 서비스 계정 JSON은 저장소 밖의 Secret mount에 보관합니다.
4. `FIREBASE_ENABLED=true`, `FIREBASE_PROJECT_ID`, `GOOGLE_APPLICATION_CREDENTIALS`를 실행 환경에 주입합니다.
5. Gateway 시작 로그와 `/actuator/health`를 확인한 뒤 테스트 페이지에서 1건을 발송합니다.

Apple APNs 연결

1. Apple Developer의 App ID에서 Push Notifications capability를 활성화합니다.
2. APNs 인증 키를 `Sandbox & Production`으로 발급하고 `.p8`, Key ID, Team ID를 안전하게 보관합니다.
3. Firebase Console의 Apple 앱 Cloud Messaging 설정에 개발 APNs 키를 등록합니다.

4. TestFlight/Ad Hoc/운영 배포 전에는 프로덕션 APNs 키 영역에도 등록합니다.
5. iOS Bundle ID, Apple Team, Firebase Apple 앱의 Bundle ID가 모두 같은지 확인합니다.

APNs 키는 Firebase와 Apple 사이의 인증 수단이며 Gateway 서버에는 배치하지 않습니다. Gateway에는 Firebase Admin 서비스 계정만 필요합니다.

```
openssl ecparam -name prime256v1 -genkey -noout -out enrollment-ec.pem
openssl pkcs8 -topk8 -nocrypt -in enrollment-ec.pem -out enrollment-key.pem
openssl pkey -in enrollment-key.pem -pubout -out enrollment-key.pem.pub
```

PUSH_QR_PRIVATE_KEY_PATH는 `enrollment-key.pem`을 가리킵니다. 같은 경로의 `.pub`도 Secret mount하고, `EnrollmentQrService.publicKeyX963Base64()`가 산출하는 공개키 Base64만 iOS Enrollment 설정에 넣습니다. 개인키는 Gateway 밖으로 내보내지 않습니다.

QR canonical은 `version|firebaseProjectId|userTopic|departmentTopic|noticeTopic|issuedAt|expiresAt|nonce`이며 `SHA256withECDSA`의 DER signature를 Base64로 넣습니다. nonce는 QR을 고유하게 할 뿐 서버가 일회 사용을 확인하는 값은 아닙니다. QR 원문과 Topic은 HTML/로그에 출력하지 않습니다.

업무 코드 연동

같은 DataSource를 사용하는 기존 `@Transactional` 업무 메서드에서 호출하면 업무 변경과 Queue INSERT가 함께 commit/rollback 됩니다.

```
pushQueueService.enqueueUser(userId, NotificationType.TASK_ARRIVED, null);
pushQueueService.enqueueDepartment(departmentId, NotificationType.SOURCE_OVERLAP, null);
pushQueueService.enqueueNotice(NotificationType.NOTICE_CREATED, null);
```

Queue는 Topic 문자열을 받지 않습니다. USER/DEPARTMENT ID는 최대 128자이고 NOTICE는 내부적으로 ALL입니다. `TASK_MENTIONED`, `COMMENT_MENTIONED`만 actorName을 요구하며 다른 타입의 actorName은 거부합니다. 제목과 본문은 `NotificationTemplateFactory`의 고정 8종만 사용합니다.

테스트 페이지와 업무 디렉터리

기본값은 비활성이라 `/internal/push-test` Bean과 URL이 존재하지 않습니다. 업무망 관리자 IP만 방화벽에서 허용하고 username과 BCrypt hash를 모두 Secret으로 설정해야 합니다. 누락되면 시작되지 않습니다. 기존 업무 인증이 있다면 in-memory 인증을 기존 인증으로 교체하되 `ROLE_PUSH_ADMIN/ROLE_PUSH_TESTER`를 유지하십시오.

```
PUSH_TEST_PAGE_ENABLED=true
PUSH_TEST_PAGE_AUTHENTICATION_ENABLED=true
PUSH_TEST_PAGE_USERNAME=<secret>
PUSH_TEST_PAGE_PASSWORD_HASH=<bcrypt-secret>
```

개발 PC의 localhost 전용 테스트에서는 `PUSH_TEST_PAGE_AUTHENTICATION_ENABLED=false`와 `SERVER_ADDRESS=127.0.0.1`로 로그인 화면을 끌 수 있습니다. 무인증 모드의 `/internal/**` 요청은 애플리케이션에서도 loopback 주소만 허용합니다. 공유·운영 환경에서는 사용하지 마십시오. 기본값은 `true`입니다.

- GET `/internal/push-test/enrollment`: 사용자·부서·공지 Topic을 구독하는 서명 QR 생성 화면
- GET `/internal/push-test/messages`: 사용자/부서 검색, 대상 선택, 고정 8종 Push 테스트 화면
- GET `/internal/push-test`: QR 등록 화면으로 이동
- GET `/`: 테스트 화면 활성화 시 로그인 후 QR 등록 화면으로 이동
- POST `/internal/push-test/send`: CSRF·권한·분당 제한·감사 로그를 거쳐 Queue 등록
- GET `/internal/push-test/events/{eventId}`: Queue/history 최소 상태
- GET `/internal/push-test/qr?userId=...&departmentId=...`: 권한 있는 관리자의 QR PNG

브라우저에서 `http://localhost:8080/`을 열면 로그인 후 QR 등록 화면으로 이동합니다. 기본 포트는 8080이며 `SERVER_PORT`로 바꿀 수 있습니다.

로컬 데모의 예시는 다음과 같습니다.

```
http://127.0.0.1:18080/internal/push-test/enrollment
http://127.0.0.1:18080/internal/push-test/messages
```

QR은 기본 3분, 최대 10분만 유효합니다. 만료됐거나 앱 공개키(Firebase Project ID가 다른 QR은 앱이 구독 전에 거부합니다.

부서·전체 공지는 확인 체크가 필수이고 성공 후 PRG redirect를 사용합니다. Topic, 서비스 계정, FCM message 원문은 화면에 노출하지 않습니다.

이 독립 저장소에는 실제 업무 사용자/부서 테이블 DDL이 없으므로 테이블을 추측해 만들지 않았습니다.

[BusinessDirectory](#) 기본 구현은 빈 결과입니다. 운영 통합 시 기존 테이블을 읽기 전용·검색어·최대 50건으로 조회하는 Bean을 제공하십시오. 사용자 결과는 `id/displayName/departmentId`, 부서는 `id/displayName`으로 매핑합니다. 본인 QR 화면은 요청 파라미터 대신 인증 Principal과 조회된 실제 부서 ID를 `EnrollmentQrService.issue()`에 전달해야 합니다.

Payload와 모니터링

FCM data는 `eventId`, `notificationType`, Gateway가 렌더링한 `title-body`, `templateVersion`을 포함합니다. 앱은 알림 문구를 자체 조립하지 않고 이 값을 그대로 저장·표시합니다. mention 타입의 `actorName`은 Queue 입력과 Gateway 렌더링에만 사용하며 FCM의 별도 필드로 보내지 않습니다. wire mapping은 `TASK_COMMENT_CREATED→COMMENT_ADDED`, `SOURCE_OVERLAP→SOURCE_CONFLICT`, `NOTICE_CREATED→NOTICE_REGISTERED`입니다.

Meter는 `push.queue.waiting`, `push.queue.waiting.by_target`, `push.queue.dead`, `push.fcm.requests`, `push.fcm.request`, `push.topic.resolution.failure`, `push.qr.issue`, `push.test.request`, `push.process.*`입니다. 사용자·부서·Topic은 label로 사용하지 않습니다.

장애 확인 순서

1. Gateway 결과가 `FCM_ACCEPTED`인지 확인합니다. 이는 Firebase 접수 성공이며 단말 표시 완료를 뜻하지 않습니다.
2. Queue가 `RETRY` 또는 `DEAD`이면 마지막 오류 분류와 서비스 계정 권한, Google outbound 443을 확인합니다.
3. iOS는 APNs development/production 입력란과 빌드 profile 환경이 맞는지 확인합니다.
4. Android/iOS 모두 앱에서 알림 권한과 개인·부서·공지 Topic 등록 완료 상태를 확인합니다.
5. 앱 재등록이 실패하면 새 QR을 발급하고 Project ID, 공개키, 기기 시각, QR TTL을 확인합니다.
6. 동일 알림이 중복되면 Gateway history의 `event_id`와 앱 로컬 저장소의 unique 처리 여부를 확인합니다.

검증

```
mvn test
```

단위 테스트는 Topic 난수/공지, actor 규칙, iOS wire payload, QR canonical/TTL/DER 서명/X9.63, retry/DEAD를 검증합니다. MVC 테스트는 기본 비활성, 인증/권한, CSRF, 관리자 PRG를 검증합니다. Docker 사용 시 Testcontainers MariaDB가 V1→V2, 동시 선점, history 원자성, stale 복구를 검증합니다. 자동 테스트는 실제 Firebase를 호출하지 않습니다.